Machine Learning

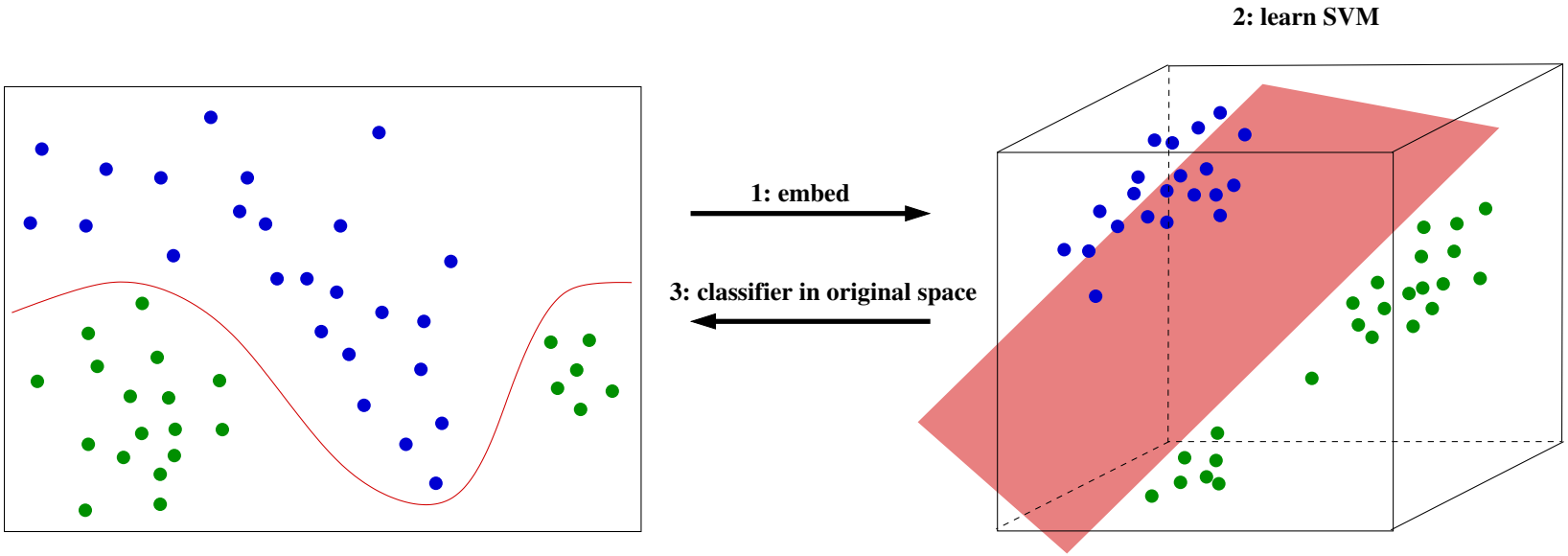
Graph Kernels

Manfred Jaeger

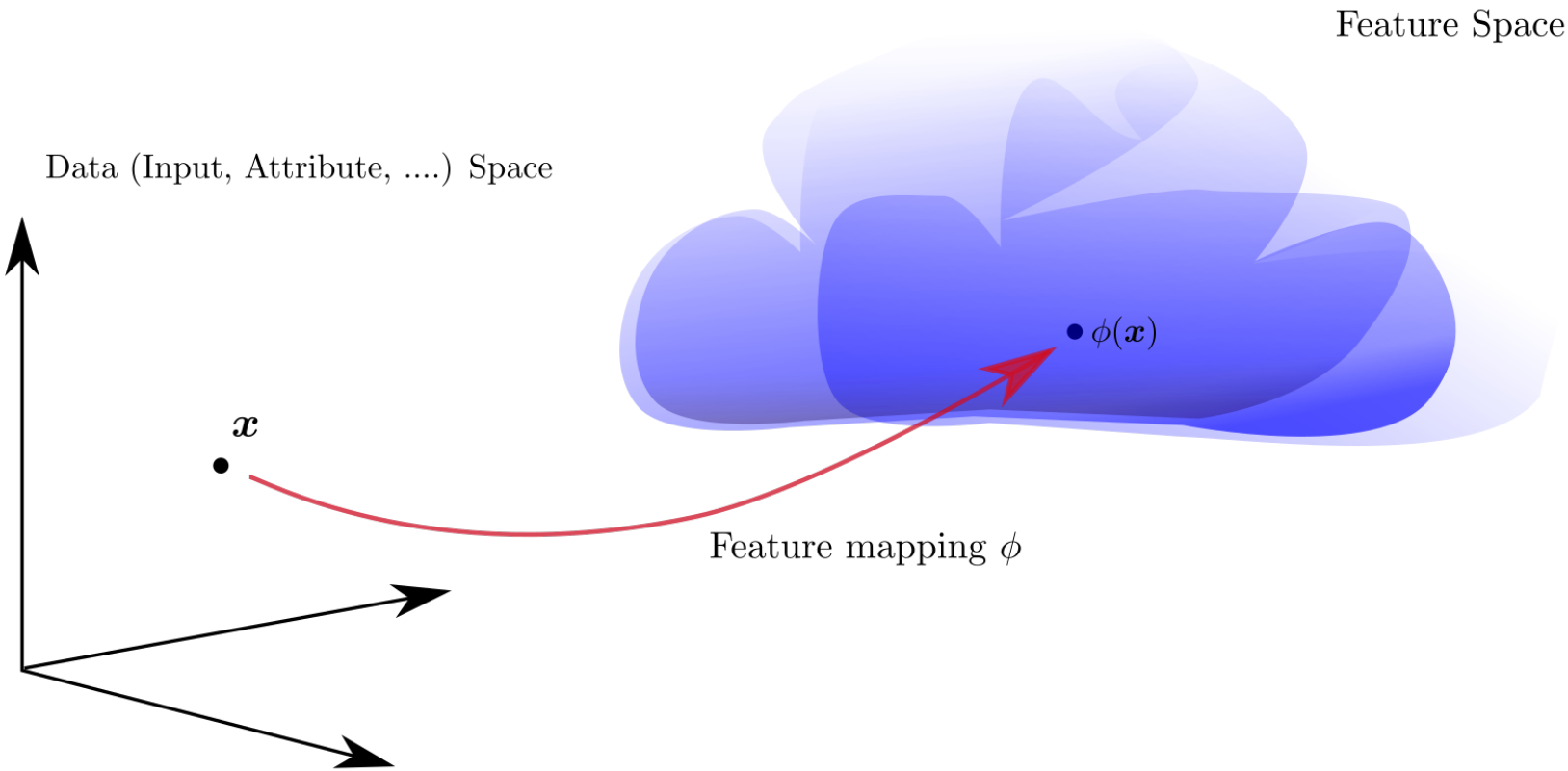
Aalborg University

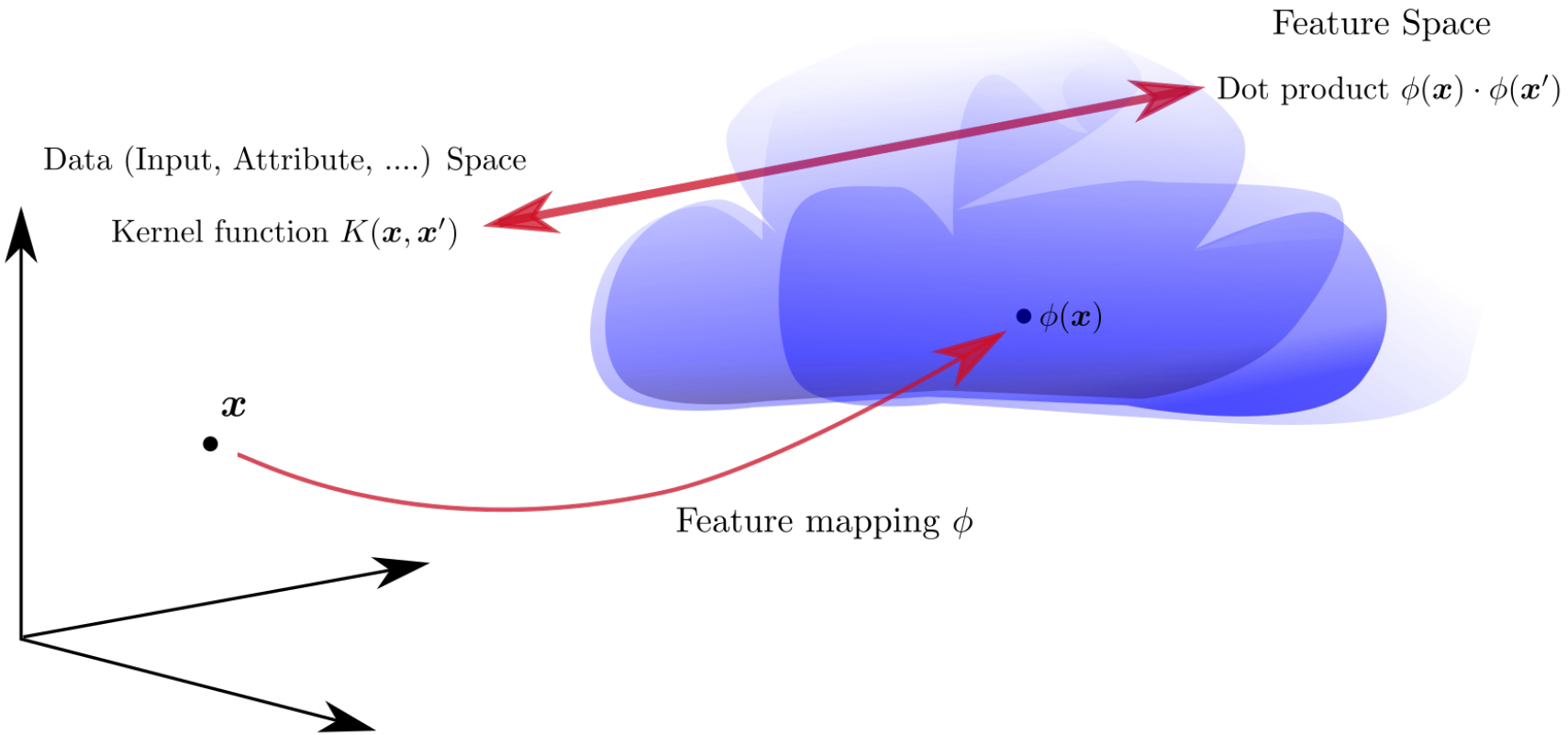
Reminder

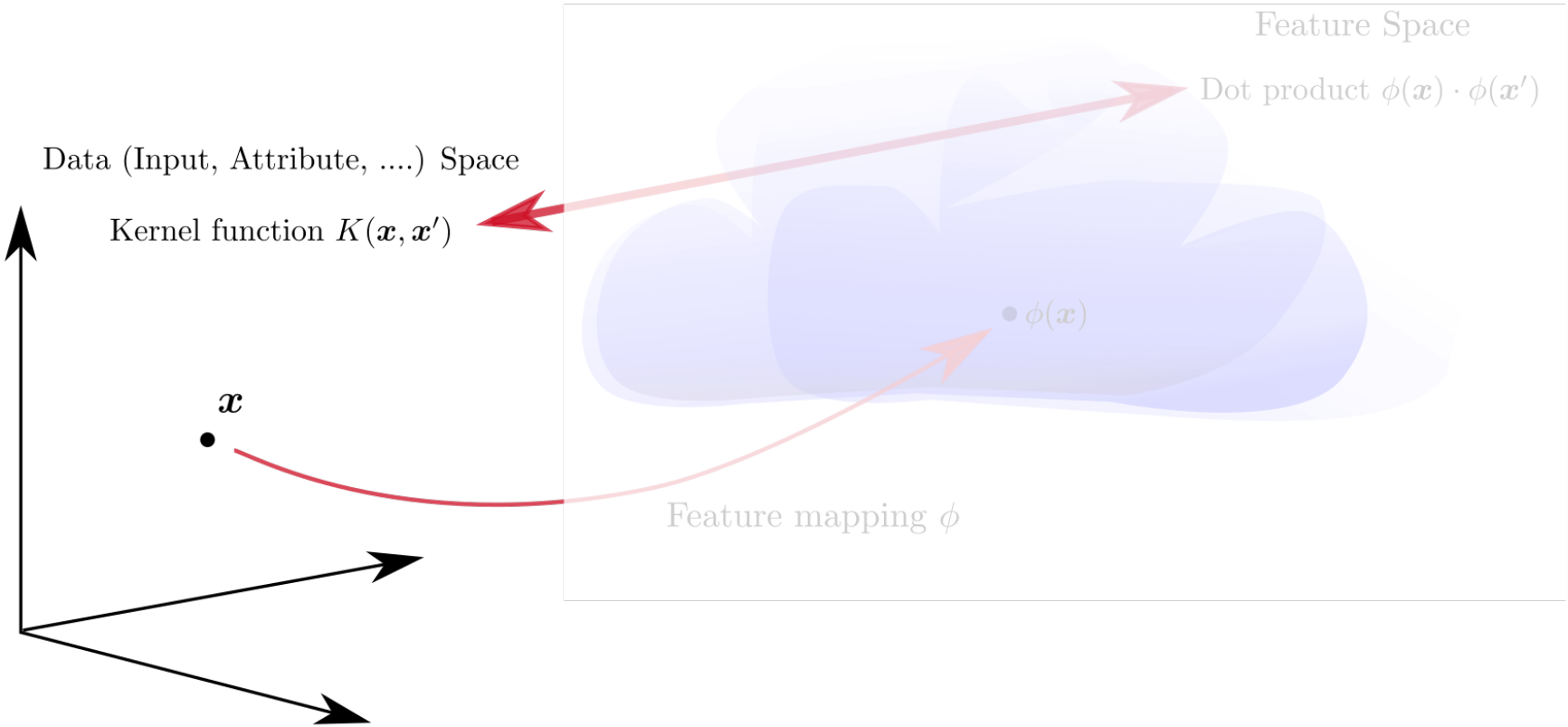
SVM learning after a non-linear transformation:



Problem: suitable feature spaces may need to be very high-dimensional. That makes computations with transformed vectors $\phi(\boldsymbol{x})$ very expensive.


Feature Space
Data (Input, Attribute, ...) Space
$\phi(\boldsymbol{x})$
$\boldsymbol{x}$
Feature mapping ϕ


Feature Space
Dot product $\phi(\boldsymbol{x}) \cdot \phi(\boldsymbol{x}')$
Data (Input, Attribute, ...) Space
Kernel function $K(\boldsymbol{x}, \boldsymbol{x}')$
$\phi(\boldsymbol{x})$
$\boldsymbol{x}$
Feature mapping ϕ



➥When we have a posititive semi-definite kernel $K(\boldsymbol{x}, \boldsymbol{x}')$ we need not know the actual feature mapping ϕ

We obtain the following strategy to construct *Kernelized* SVM classifiers:

- **Given:** a classification problem
- **Define** or **Select** a similarity function

$$K(\boldsymbol{x}, \boldsymbol{z})$$

- **Verify** that $K(\boldsymbol{x}, \boldsymbol{z})$ is positive semi-definite
- **Learn** an SVM using the values $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ in place of dot products $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$.

➥The learning algorithm requires as input "only" the kernel matrix for the training datapoints $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$.

We obtain the following strategy to construct *Kernelized* SVM classifiers:

- **Given:** a classification problem
- **Define** or **Select** a similarity function
$$K(\boldsymbol{x}, \boldsymbol{z})$$
- **Verify** that $K(\boldsymbol{x}, \boldsymbol{z})$ is positive semi-definite
- **Learn** an SVM using the values $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ in place of dot products $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$.

➥The learning algorithm requires as input "only" the kernel matrix for the training datapoints $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$.

➥This also opens the SVM technology to datatypes other than numeric data: just need to find kernels: positive semi-definite (similarity) functions on pairs of data objects.

Given: a set of graphs $\mathcal{G}$, e.g.

- $\mathcal{G}$: set of all finite undirected graphs (V, E)
- $\mathcal{G}$: set of all finite directed graphs (V, E, A_1, A_2) with node attributes A_1, A_2.
- ...

Want: a feature function

$$\phi : G \mapsto \phi(G) \in \mathbb{R}^d \quad (G \in \mathcal{G})$$

(explicit construction), **or** a posititive semi-definite kernel

$$K : (G_0, G_1) \mapsto K(G_0, G_1) \in \mathbb{R} \quad (G_0, G_1 \in \mathcal{G})$$

(implicit construction).

Convolution Kernels

Let $\mathcal{D}$ be a (structured) data type (e.g.: $\mathcal{D} = \mathcal{G}$ a type of graphs).

Decompositions

- $\mathcal{P}$: a space of "components" or "parts" of objects from $\mathcal{D}$
- $\mathcal{R}$: a relation between parts and objects with the interpretation: $(p, d) \in \mathcal{R}$ if p is a part of d.
- Define $\mathcal{R}^{-1}(d) := \{p | (p, d) \in \mathcal{R}\}$ (the parts of d)

Convolution Kernels

Let K_P be a Kernel on $\mathcal{P}$. Then

$$K_D(d, d') = \sum_{p \in \mathcal{R}^{-1}(d)} \sum_{p' \in \mathcal{R}^{-1}(d')} K_P(p, p')$$

is a Kernel on $\mathcal{D}$.

Notes:

- more complex decompositions into different types of parts are usually considered
- the $\mathcal{R}$ relation is very abstract and need not correspond to an intuitive "part of" relationship.

[Haussler, David. Convolution kernels on discrete structures. Vol. 646. Technical report, Department of Computer Science, University of California at Santa Cruz, 1999.]

Dot product as convolution kernel

$\mathcal{D} = \mathbb{R}^n$, $\mathcal{P} = \{1, \ldots, n\} \times \mathbb{R}$.

$$((i, z), \boldsymbol{x})) \in \mathcal{R} \Leftrightarrow \boldsymbol{x}[i] = z.$$

Parts kernel:

$$K_P((i, y), (j, z)) = \begin{cases} y \cdot z & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Then

$$K_D(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x} \cdot \boldsymbol{y}.$$

Dot product as convolution kernel

$\mathcal{D} = \mathbb{R}^n$, $\mathcal{P} = \{1, \ldots, n\} \times \mathbb{R}$.

$$((i, z), \boldsymbol{x})) \in \mathcal{R} \Leftrightarrow \boldsymbol{x}[i] = z.$$

Parts kernel:

$$K_P((i, y), (j, z)) = \begin{cases} y \cdot z & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Then

$$K_D(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x} \cdot \boldsymbol{y}.$$

Vectors of different lengths

$\mathcal{D} = \bigcup_{n \in \mathbb{N}} \mathbb{R}^n$, $\mathcal{P} = \mathbb{N} \times \mathbb{R}$.

$$((i, z), \boldsymbol{x})) \in \mathcal{R} \Leftrightarrow \boldsymbol{x}[i] = z.$$

Parts kernel:

$$K_P((i, y), (j, z)) = y \cdot z.$$

Then for $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{y} \in \mathbb{R}^m$:

$$K_D(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i=1}^{n} \sum_{j=1}^{m} \boldsymbol{x}[i] \cdot \boldsymbol{y}[j].$$

➥Convolution kernels enable comparison of objects of different sizes (without defining an "alignment" of parts).

We consider $\mathcal{D} = \Sigma^*$ for some alphabet Σ.

String Kernel 1

Define $\mathcal{P} = \Sigma^n$ ("n-grams") and for $\boldsymbol{u} \in \mathcal{P}$, $\boldsymbol{s} \in \mathcal{D}$:

$$(\boldsymbol{u}, \boldsymbol{s}) \in \mathcal{R} \Leftrightarrow \boldsymbol{u} \text{ appears as a substring in } \boldsymbol{s}.$$

Parts kernel:

$$K_P(\boldsymbol{u}, \boldsymbol{v}) = \begin{cases} 0 & \text{if } \boldsymbol{u} \neq \boldsymbol{v} \\ 1 & \text{if } \boldsymbol{u} = \boldsymbol{v} \end{cases} \quad (\boldsymbol{u}, \boldsymbol{v} \in \mathcal{P})$$

Then:

$$K_D(\boldsymbol{s}, \boldsymbol{t}) = |\{\boldsymbol{u} \in \Sigma^n | (\boldsymbol{u}, \boldsymbol{s}) \in \mathcal{R} \text{ and } (\boldsymbol{u}, \boldsymbol{t}) \in \mathcal{R}\}|$$

(number of common n-grams of $\boldsymbol{s}$ and $\boldsymbol{t}$).

String Kernel 2

Define $\mathcal{P} = \mathbb{N} \times \Sigma^n$ and for $(k, \boldsymbol{u}) \in \mathcal{P}$, $\boldsymbol{s} \in \mathcal{D}$:

$$((k, \boldsymbol{u}), \boldsymbol{s}) \in \mathcal{R} \Leftrightarrow \boldsymbol{u} \text{ is the substring of length } n \text{ starting at index } k \text{ in } \boldsymbol{s}.$$

Parts kernel:

$$K_P((k, \boldsymbol{u}), (l, \boldsymbol{v})) = \begin{cases} 0 & \text{if } \boldsymbol{u} \neq \boldsymbol{v} \\ 1 & \text{if } \boldsymbol{u} = \boldsymbol{v} \end{cases} \quad ((k, \boldsymbol{u}), (l, \boldsymbol{v}) \in \mathcal{P})$$

Then:

$$K_D(\boldsymbol{s}, \boldsymbol{t}) = \sum_{\boldsymbol{u} \in \Sigma^n} |\{k \in \mathbb{N} | ((k, \boldsymbol{u}), \boldsymbol{s}) \in \mathcal{R}\}| \cdot |\{k \in \mathbb{N} | ((k, \boldsymbol{u}), \boldsymbol{t}) \in \mathcal{R}\}|$$

➥This is the n-spectrum kernel!

$\mathcal{D}$: finite graphs (V, E, L) with a node label attribute L.
$\mathcal{P}$: set of nodes in graphs from $\mathcal{D}$.

$$(v, (V, E, L)) \in \mathcal{R} \Leftrightarrow v \in V$$

Parts kernel: kernel on L, e.g. 0/1 for categorical L, product for numerical L.

Then:

$$K_D((V, E, L), (V', E', L')) = \sum_{v \in V} \sum_{v' \in V'} K_P(L(v), L(v')).$$

➥Graph structures given by E, E' not taken into account!

Sub-graphs and Isomorphisms

Encoding E for a graph with vertices $V = \{1, \ldots, n\}$:

Adjacency List

Directed:
list the targets of outgoing edges of a node:

1:	2
2:	6
3:	
4:	2,6
5:	1,6
6:	3

Undirected:
list all neighbors of a node:

1:	2,3,5
2:	1,3
3:	1,2
4:	6
5:	1,6
6:	4,5

Adjacency Matrix

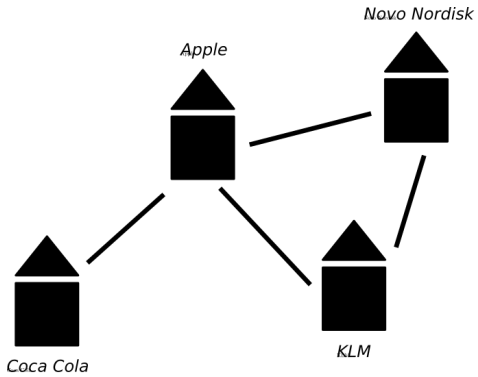
For each pair of vertices: indicate whether edge exists

Directed:

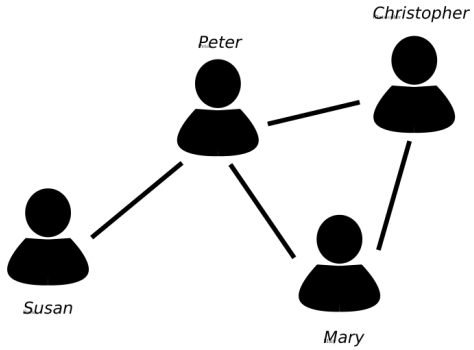
$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Undirected: symmetric matrix

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

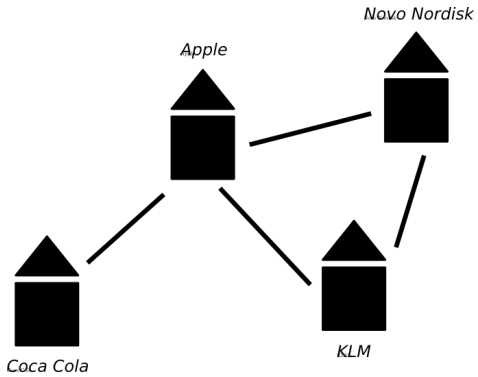


$G_1 = (V_1, E_1)$

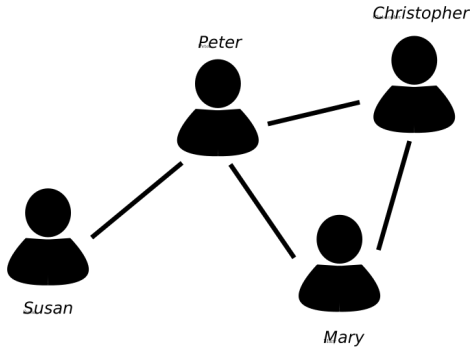


$G_2 = (V_2, E_2)$

Are these two graphs identical?



$G_1 = (V_1, E_1)$



$G_2 = (V_2, E_2)$

Are these two graphs identical?

No, because

$$V_1 = \{\textit{Coca Cola}, \textit{Apple}, \textit{Novo Nordisk}, \textit{KLM}\} \neq V_2 = \{\textit{Susan}, \textit{Peter}, \textit{Christopher}, \textit{Mary}\}$$

Isomorphic Graphs

$(V_1, E_1), (V_2, E_2)$ are *isomorphic*, if there exists a bijective mapping

$$f : V_1 \to V_2$$

such that for all $v, v' \in V_1$:

$$(v, v') \in E_1 \Leftrightarrow (f(v), f(v')) \in E_2.$$

The function f then is called a *graph isomorphism*.

Isomorphic Graphs

$(V_1, E_1), (V_2, E_2)$ are *isomorphic*, if there exists a bijective mapping

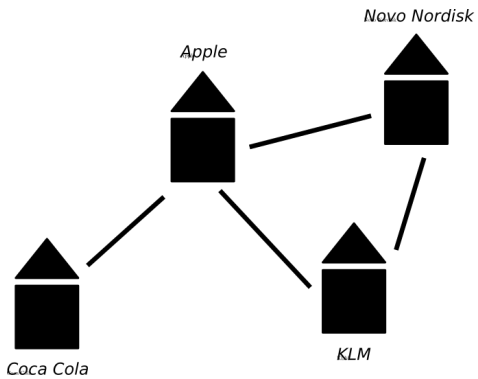
$$f : V_1 \to V_2$$

such that for all $v, v' \in V_1$:

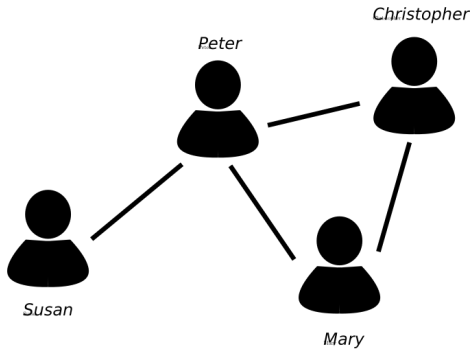
$$(v, v') \in E_1 \Leftrightarrow (f(v), f(v')) \in E_2.$$

The function f then is called a *graph isomorphism*.

Example



$G_1 = (V_1, E_1)$



$G_2 = (V_2, E_2)$

are isomorphic, as witnessed by:

$$f : \textit{Coca Cola} \mapsto \textit{Susan}, \textit{Apple} \mapsto \textit{Peter}, \textit{Novo Nordisk} \mapsto \textit{Christopher}, \textit{KLM} \mapsto \textit{Mary}$$

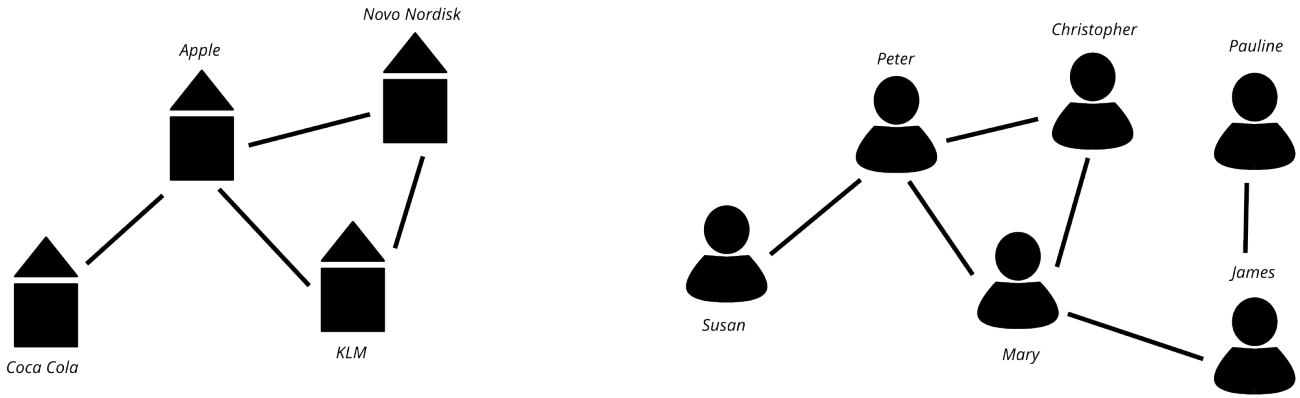
Question: is this the only isomorphism?

(V_1, E_1) is *subgraph-isomorphic* to (V_2, E_2), if there exists an injective mapping

$$f : V_1 \to V_2$$

such that for all $v, v' \in V_1$:

$$(v, v') \in E_1 \Leftrightarrow (f(v), f(v')) \in E_2.$$

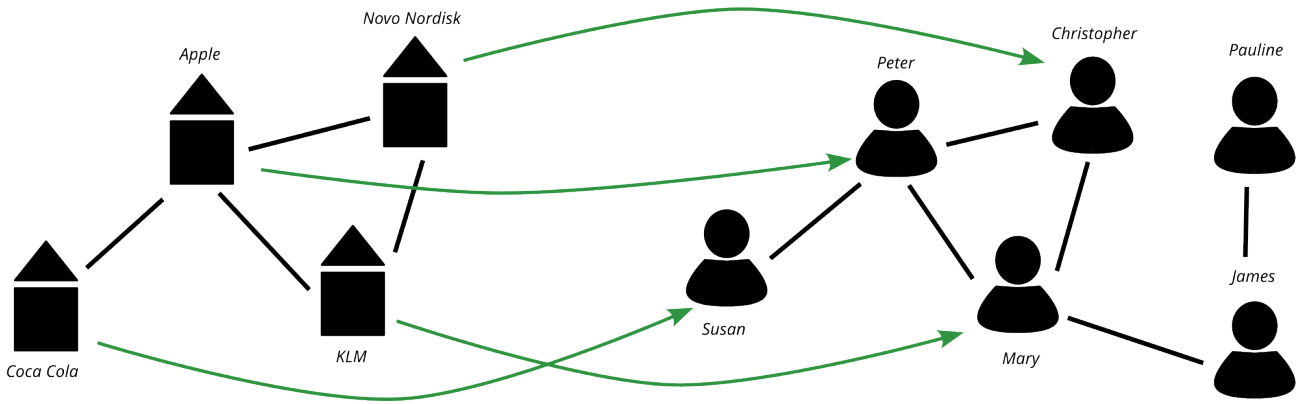


(V_1, E_1) is *subgraph-isomorphic* to (V_2, E_2), if there exists an injective mapping

$$f : V_1 \to V_2$$

such that for all $v, v' \in V_1$:

$$(v, v') \in E_1 \Leftrightarrow (f(v), f(v')) \in E_2.$$

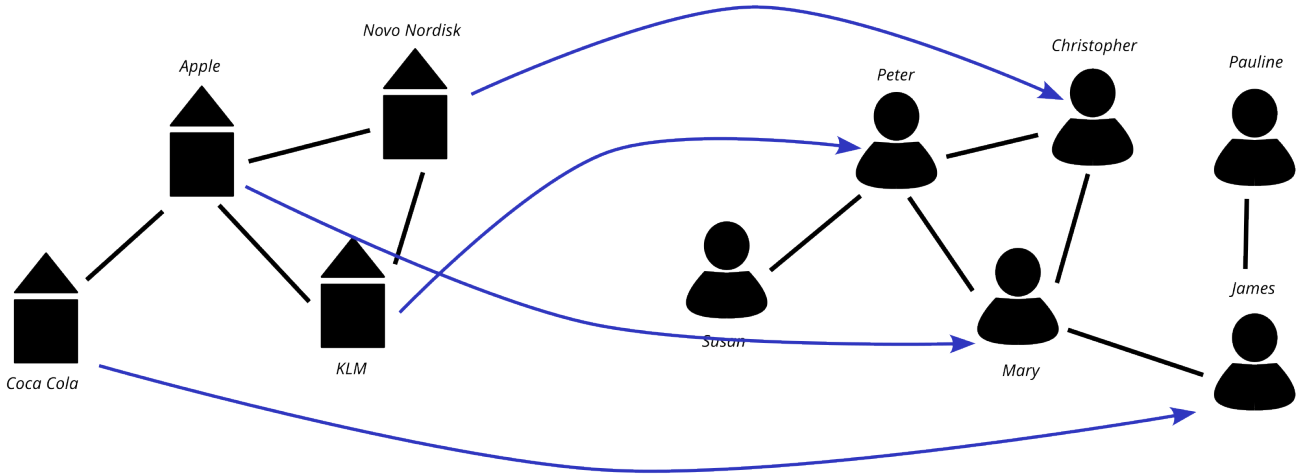


(V_1, E_1) is *subgraph-isomorphic* to (V_2, E_2), if there exists an injective mapping

$$f : V_1 \to V_2$$

such that for all $v, v' \in V_1$:

$$(v, v') \in E_1 \Leftrightarrow (f(v), f(v')) \in E_2.$$



Problem

Given graphs $(V_1, E_1), (V_2, E_2)$ (as adjacency matrices, adjacency lists, ...), is (V_1, E_1) (sub-graph) isomorphic to (V_2, E_2)?

Answer

Can check each of the

$$\frac{|V_2|!}{(|V_2| - |V_1|)!}$$

possible injective mappings $f : V_1 \to V_2$.

➥This is infeasible for large graphs (exponential in $|V_1|$)!

Open Problem

Is there an efficient way to decide whether (V_1, E_1) is an isomorphic subgraph of (V_2, E_2)?

➥Probably not: this is an NP-hard problem!

Graphlet Kernel

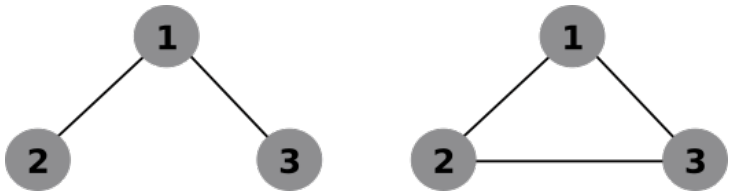
For simplicity: consider undirected graphs without attributes.

Graphlet

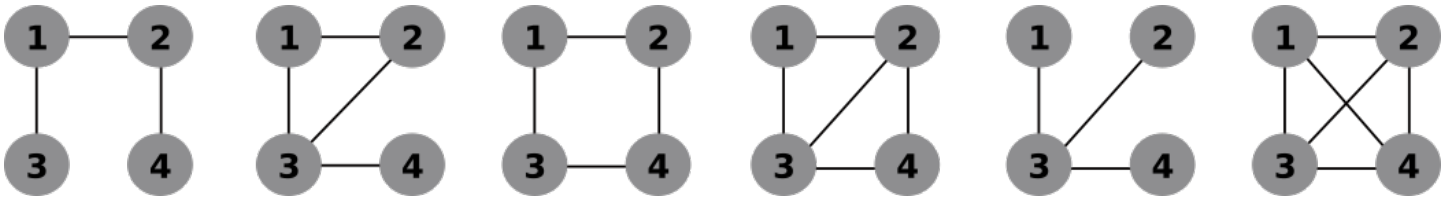
Graphlet: a small graph (e.g. size 3,4,5). We consider collections $\mathcal{G}$ of graphlets, where usually

- all $g \in \mathcal{G}$ have the same size
- only consider connected g
- at most one representative of each isomorphism class included in $\mathcal{G}$

Size 3 graphlets:



Size 4 graphlets:



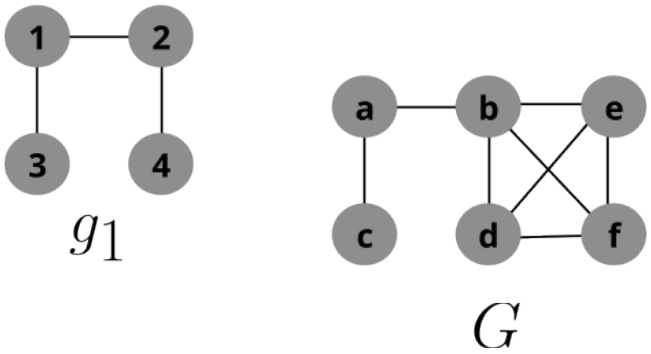
➥similar to n-grams for strings

[Shervashidze, Nino, et al. "Efficient graphlet kernels for large graph comparison." Artificial intelligence and statistics. PMLR, 2009.]

Graphlet features

Graphlet g defines feature ϕ_g of (bigger) graph G: number of occurrences of g as a subgraph of G.

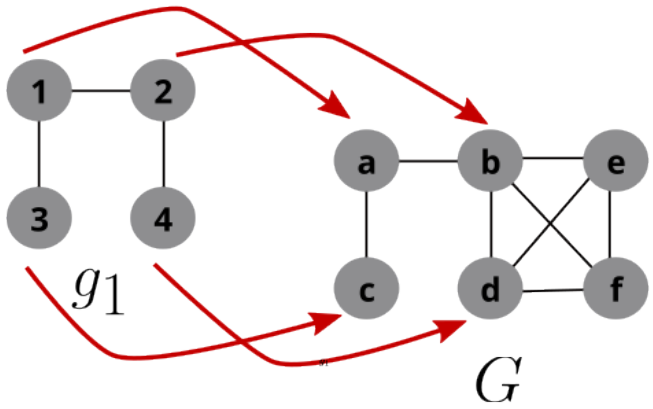
Version 1: count the number of isomorphisms that map g to an induced subgraph of G:



Graphlet features

Graphlet g defines feature ϕ_g of (bigger) graph G: number of occurrences of g as a subgraph of G.

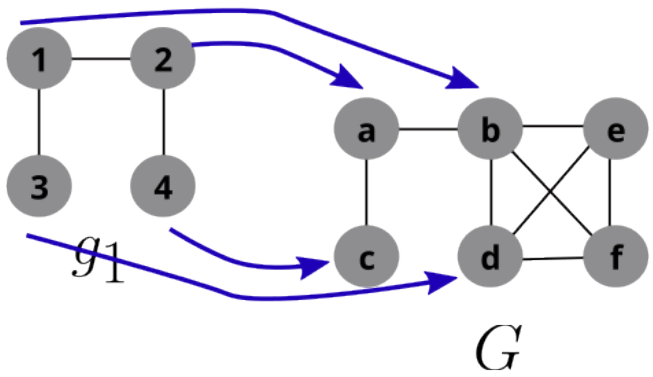
Version 1: count the number of isomorphisms that map g to an induced subgraph of G:



Graphlet features

Graphlet g defines feature ϕ_g of (bigger) graph G: number of occurrences of g as a subgraph of G.

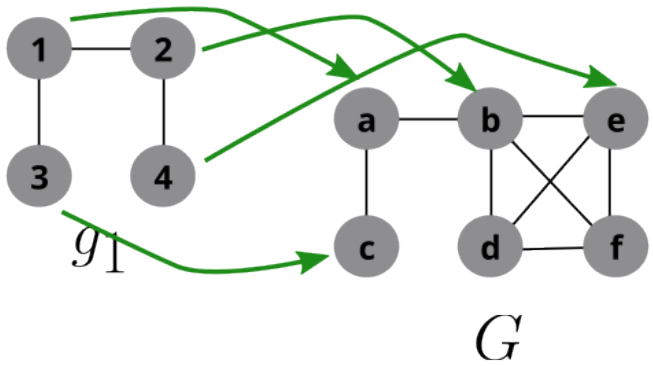
Version 1: count the number of isomorphisms that map g to an induced subgraph of G:



Graphlet features

Graphlet g defines feature ϕ_g of (bigger) graph G: number of occurrences of g as a subgraph of G.

Version 1: count the number of isomorphisms that map g to an induced subgraph of G:

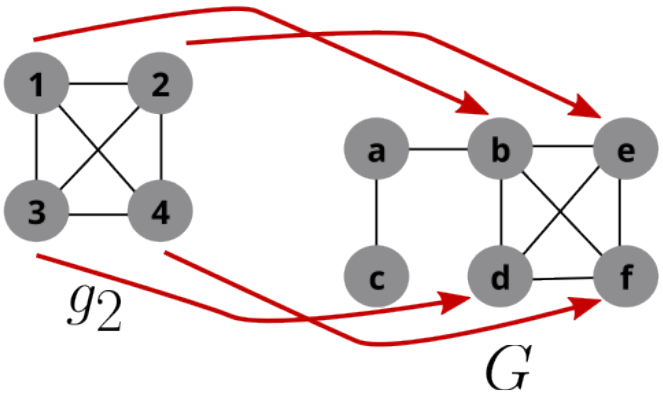


6 different isomorphisms embed g_1 in G: $\phi^{(1)}_{g_1}(G) = 6$

Graphlet features

Graphlet g defines feature ϕ_g of (bigger) graph G: number of occurrences of g as a subgraph of G.

Version 1: count the number of isomorphisms that map g to an induced subgraph of G:

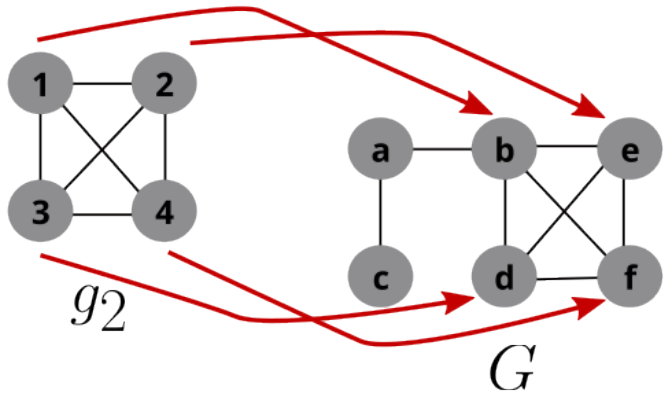


4!=24 different isomorphisms embed g_2 in G: $\phi^{(1)}_{g_2}(G) = 24$

Graphlet features

Graphlet g defines feature ϕ_g of (bigger) graph G: number of occurrences of g as a subgraph of G.

Version 1: count the number of isomorphisms that map g to an induced subgraph of G:



4!=24 different isomorphisms embed g_2 in G: $\phi^{(1)}_{g_2}(G) = 24$

Version 2: only count how many induced subgraphs are isomorphic to $g \in \mathcal{G}$:
$\phi^{(2)}_{g_1}(G) = 3, \phi^{(2)}_{g_2}(G) = 1$.

Raw count feature vectors $\phi^{(1)}$ or $\phi^{(2)}$ have larger entries for larger graphs.

➥according to the corresponding kernels $K^{(1)}, K^{(2)}$, larger graphs will be "more similar" to other graphs than smaller graphs.

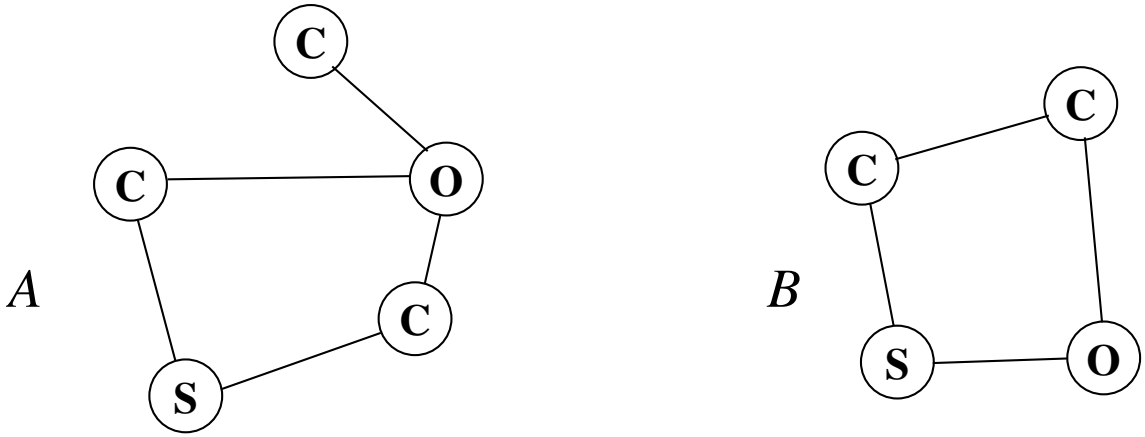
Solution 1: normalize the feature vectors ϕ to probability distributions.

Solution 2: apply the generic kernel normalization (normalize feature vectors to length 1):

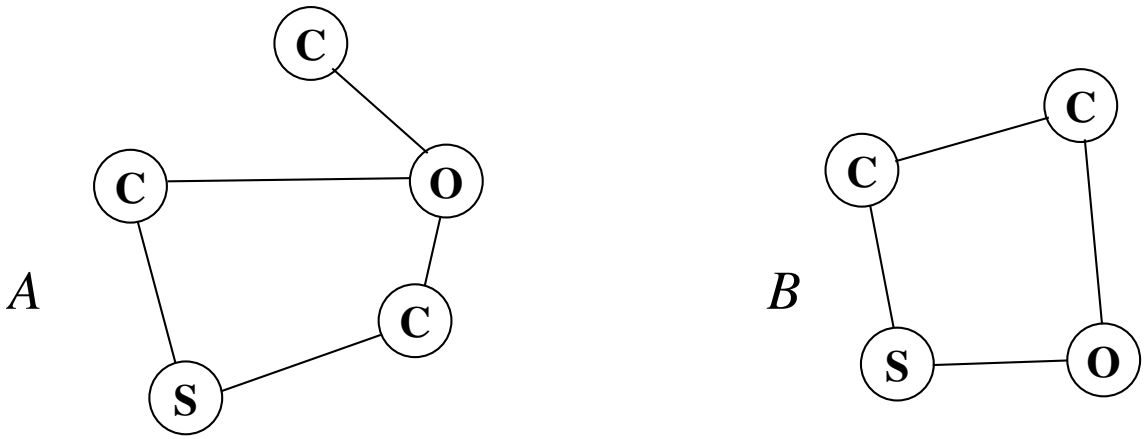
$$K(G, G') \to \frac{K(G, G')}{\sqrt{K(G, G) \cdot K(G', G')}}.$$

Random Walk Kernels

Node-labeled graphs (there could also be edge labels):



Node-labeled graphs (there could also be edge labels):



Random walk:

- Pick a random start vertex (e.g. uniform)
- make random transitions (e.g. uniformly to any neighbor)
- terminate random walk at any step with a fixed termination probability p_t

A

Label sequence	Probability
C	$\frac{3}{5}p_t$
O	$\frac{1}{5}p_t$
CO	$\frac{1}{5}\frac{1}{1}p_t + \frac{1}{5}\frac{1}{2}p_t + \frac{1}{5}\frac{1}{2}p_t$
...	...

B

Label sequence	Probability
C	$\frac{2}{4}p_t$
O	$\frac{1}{4}p_t$
CO	$\frac{1}{4}\frac{1}{2}p_t$
...	...

Label sequence kernel

Let Σ be the alphabet of node labels. The random walk model defines for each graph G a probability distribution p_G over Σ^*. This is the feature vector for the *label sequence kernel*:

$$K(G, G') = \sum_{\boldsymbol{s} \in \Sigma^*} p_G(\boldsymbol{s}) p_{G'}(\boldsymbol{s})$$

"Efficient" computation of this infinite sum possible in $O(n^4)$ by dynamic programming (n: max. number of nodes in G or G')!

Adding string kernel

Taking similarity of distinct label sequences into account:

$$K(G, G') = \sum_{\boldsymbol{s} \in \Sigma^*} \sum_{\boldsymbol{t} \in \Sigma^*} K(\boldsymbol{s}, \boldsymbol{t}) p_G(\boldsymbol{s}) p_{G'}(\boldsymbol{t})$$

where $K(\boldsymbol{s}, \boldsymbol{t})$ could be any string kernel.

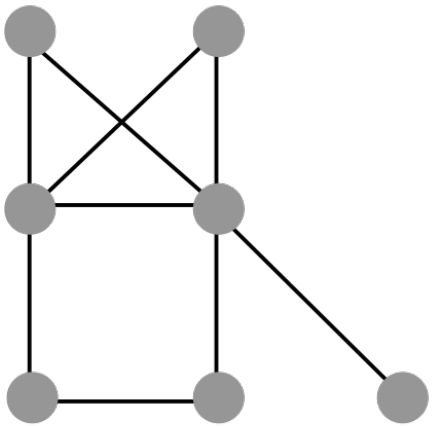
Including edge labels

Random walk kernels easily incorporate edge labels: random walks then are sequences of alternating node and edge labels.

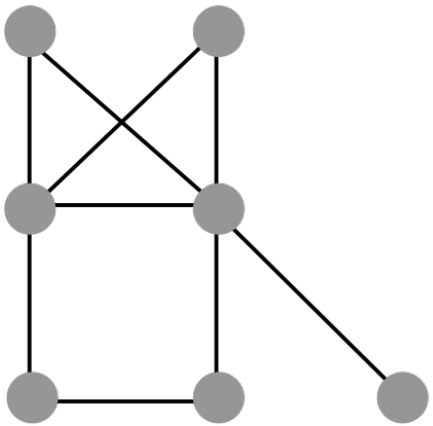
[Tsuda,Saigo: Graph Classification. *In:* Aggarwal et al.: Managing and Mining Graph Data. Springer 2010]

Weisfeiler-Lehman Kernel

W-L node code refinement process for (undirected) graph (V, E)

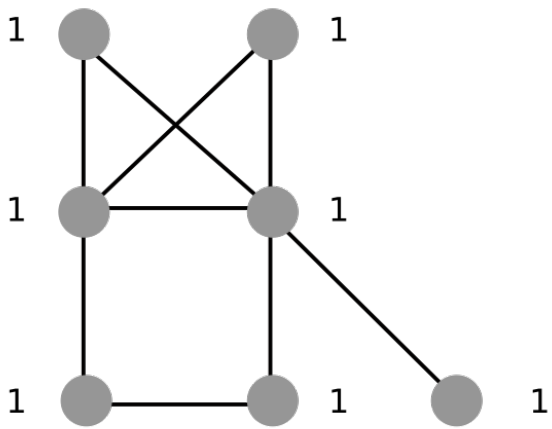


W-L node code refinement process for (undirected) graph (V, E)



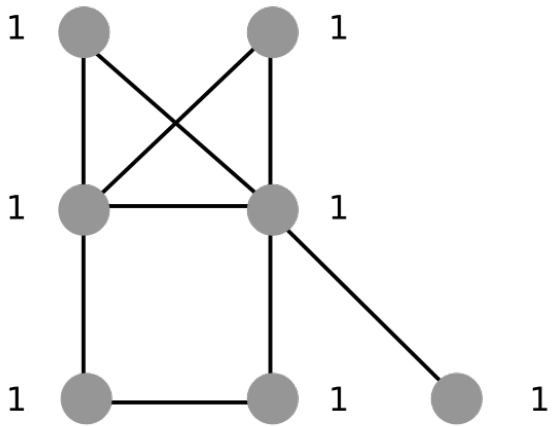
Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

W-L node code refinement process for (undirected) graph (V, E)



Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

W-L node code refinement process for (undirected) graph (V, E)



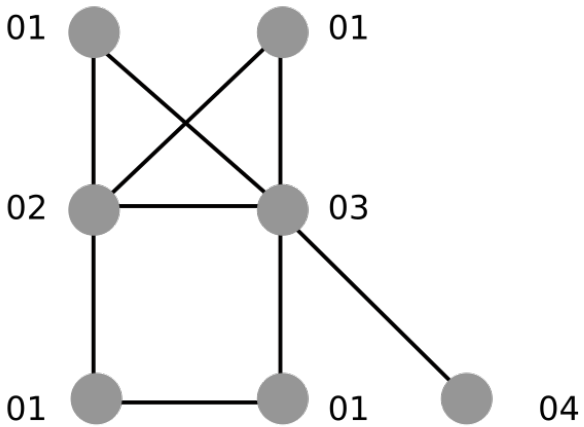
Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

In iteration k: compute code

$$c^k(v) = \text{hash}((c^{k-1}(v), \{\!\{c^{k-1}(w) : (v, w) \in E\}\!\}))$$

where hash is a function that takes as input a pair consisting of a code and a multiset of codes, and returns a unique new code.

W-L node code refinement process for (undirected) graph (V, E)



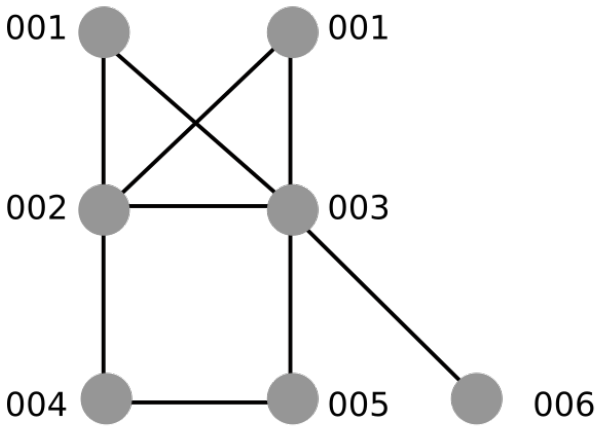
Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

In iteration k: compute code

$$c^k(v) = \text{hash}((c^{k-1}(v), \{\!\{c^{k-1}(w) : (v, w) \in E\}\!\}))$$

where hash is a function that takes as input a pair consisting of a code and a multiset of codes, and returns a unique new code.

W-L node code refinement process for (undirected) graph (V, E)



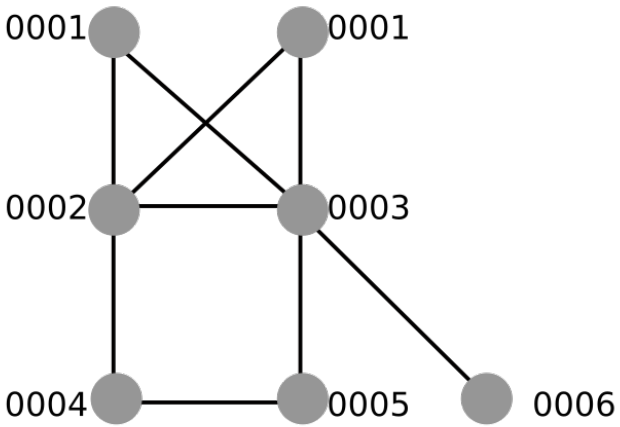
Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

In iteration k: compute code

$$c^k(v) = \text{hash}((c^{k-1}(v), \{\!\{c^{k-1}(w) : (v, w) \in E\}\!\}))$$

where hash is a function that takes as input a pair consisting of a code and a multiset of codes, and returns a unique new code.

W-L node code refinement process for (undirected) graph (V, E)



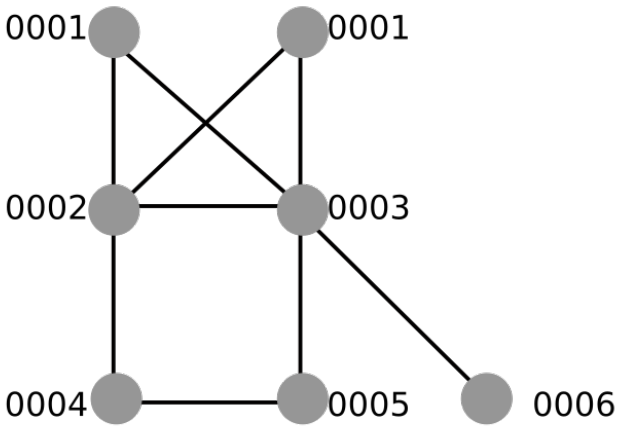
Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

In iteration k: compute code

$$c^k(v) = \text{hash}((c^{k-1}(v), \{\!\{c^{k-1}(w) : (v, w) \in E\}\!\}))$$

where hash is a function that takes as input a pair consisting of a code and a multiset of codes, and returns a unique new code.

W-L node code refinement process for (undirected) graph (V, E)



Initialize node codes c^0: node attribute vector if exists; otherwise a constant code

In iteration k: compute code

$$c^k(v) = \text{hash}((c^{k-1}(v), \{\!\{c^{k-1}(w) : (v, w) \in E\}\!\}))$$

where hash is a function that takes as input a pair consisting of a code and a multiset of codes, and returns a unique new code.

- At iteration k the codes define a partition P^k of the nodes into sets of nodes with equal codes
- P^k is a refinement of P^{k-1}
- If $P^k = P^{k-1}$, then $P^{k+l} = P^{k-1}$ for all $l \geq 0$
- After at most $|V|$ iterations the partition becomes stable

Let $G = (V, E), G' = (V', E')$ be isomorphic graphs with isomorphism $I : V \to V'$. Then for all $v \in V, k \geq 0$:

$$c^k(I(v)) = c^k(v)$$

(proof by induction on k). In particular:

$$\{\!\{c^k(v) : v \in V\}\!\} = \{\!\{c^k(v') : v' \in V'\}\!\}$$

Let $G = (V, E), G' = (V', E')$ be isomorphic graphs with isomorphism $I : V \to V'$. Then for all $v \in V, k \geq 0$:

$$c^k(I(v)) = c^k(v)$$

(proof by induction on k). In particular:

$$\{\!\{c^k(v) : v \in V\}\!\} = \{\!\{c^k(v') : v' \in V'\}\!\}$$

WL-test for isomorphism: given graphs G, G' with $n = |V| = |V'|$:

- run WL code updates for n iterations;
- if $\{\!\{c^n(v) : v \in V\}\!\} \neq \{\!\{c^n(v') : v' \in V'\}\!\}$: return "not isomorphic"; otherwise: return "don't know".

Let $G = (V, E), G' = (V', E')$ be isomorphic graphs with isomorphism $I : V \to V'$. Then for all $v \in V, k \geq 0$:

$$c^k(I(v)) = c^k(v)$$

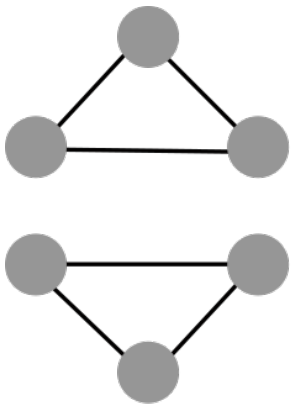
(proof by induction on k). In particular:

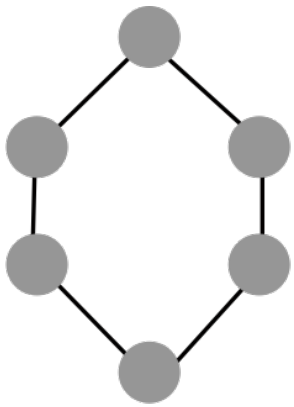
$$\{\!\{c^k(v) : v \in V\}\!\} = \{\!\{c^k(v') : v' \in V'\}\!\}$$

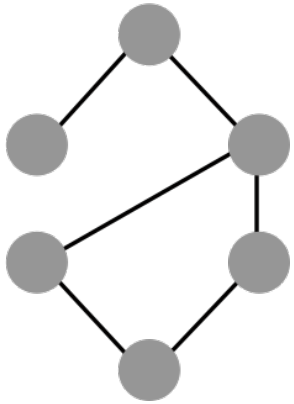
WL-test for isomorphism: given graphs G, G' with $n = |V| = |V'|$:

- run WL code updates for n iterations;
- if $\{\!\{c^n(v) : v \in V\}\!\} \neq \{\!\{c^n(v') : v' \in V'\}\!\}$: return "not isomorphic"; otherwise: return "don't know".

Exercise: which of the following graphs will be identified as non-isomorphic by the WL-test?







Let $\mathcal{C}^k$ be the code space for codes generated in the k'th iteration (for all possible graphs).

Each $\kappa \in \mathcal{C}^k$ defines a feature for graphs $G = (V, E)$:

$$\phi^{\kappa}(G) = |\{v \in V : c^k(v) = \kappa\}|$$

Fix $h \geq 0$. The **WL-kernel** of depth h is defined by the features

$$\{\phi^{\kappa} : \kappa \in \mathcal{C}^k, k = 0, \ldots, h\}$$

➥infinite dimensional feature vectors ($\mathcal{C}^k$ infinite for $k \geq 1$), but for any graph G only finitely many $\phi^{\kappa}(G)$ are nonzero.

➥more precisely, this is the WL **subtree kernel**. There are some other types of WL-kernels.

Kernels for node classification

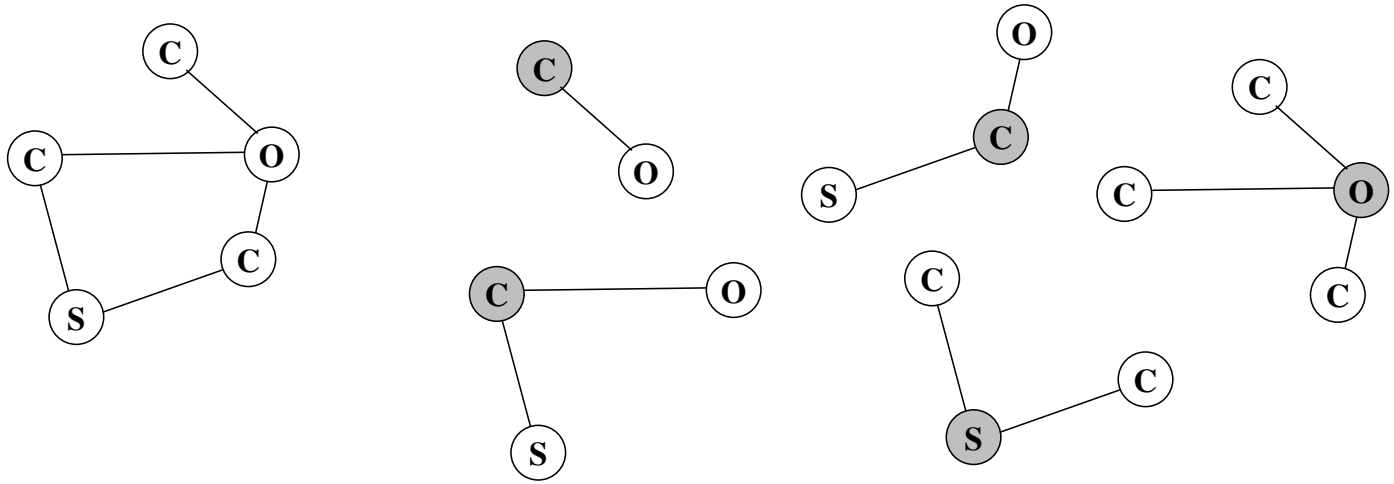
If we want a kernel for nodes, rather than full graphs:

Node centered random walks

Feature vector for node $v \in G$ given by label sequence probabilities for random walks starting at v

Ego graphs

For any graph kernel $k(G, G')$: can apply kernel to the *ego graphs* of nodes v (subgraph consisting of v and its immediate neighbors).



➥beware: the node of interest has no special role in the application of $k(G, G')$ to the ego graphs